

Handling Forms via Form Actions

Easy Guide

Contents

1	What are Form Actions?	2
1.1	Understanding Form Actions	2
1.1.1	The Old Way vs New Way	2
1.2	What is a Form Action Function?	2
1.2.1	Key Benefit	3
1.3	Where Form Actions Work	3
2	Adding Validation Checks	4
2.1	Validation in Form Actions	4
2.2	Where to Validate	4
2.3	Common Validations	4
2.4	Steps for Validation	4
2.4.1	Example Validation Check	5
2.5	Returning Validation Errors	5
3	Managing Form-dependent State with useActionState()	6
3.1	What is useActionState()?	6
3.2	What It Returns	6
3.3	Basic Usage	6
3.4	How It Works	6
3.5	Displaying Errors	7
4	Using User Input	8
4.1	Accessing Form Data	8
4.2	FormData Object	8
4.2.1	Extracting Data	8
4.3	Field Names Matter	8
4.4	Getting Multiple Values	8
4.5	File Uploads	9
5	Moving the Action Function out of the Component	10
5.1	Why Move the Action?	10
5.2	How to Organize	10
5.3	Creating Action File	10
5.4	Using in Component	11
5.5	The 'use server' Directive	11

6	Handling Form Submission	12
6.1	Form Submission Flow	12
6.2	Preventing Default Behavior	12
6.3	After Submission	12
6.4	Example: Complete Flow	13
7	Working with Asynchronous Form Actions	14
7.1	Why Async?	14
7.2	Using Async Syntax	14
7.3	Waiting for Results	14
7.4	Handling Errors	15
7.5	Multiple Operations	15
8	Updating the UI with useFormStatus()	16
8.1	What is useFormStatus()?	16
8.2	What It Returns	16
8.3	Common Use: Disable Submit Button	16
8.4	Show Loading Spinner	17
8.5	Disable All Form Fields	17
8.6	Important Note	17
9	Registering Multiple Form Actions	18
9.1	One Form, Multiple Actions	18
9.2	Using formAction Attribute	18
9.3	Different Actions	18
9.4	Detecting Which Button	19
10	Sending an HTTP Request via a Form Action	20
10.1	Making API Calls	20
10.2	Using Fetch	20
10.3	Error Handling	21
10.4	Different HTTP Methods	21
11	Using the "pending" State from useActionState()	22
11.1	Accessing Pending State	22
11.2	What isPending Tells You	22
11.3	Common Uses	22
11.3.1	Disable Submit Button	22
11.3.2	Show/Hide Spinner	22
11.3.3	Disable Entire Form	22
11.3.4	Show Different Message	23
11.4	Pending vs useFormStatus	23
12	Adding Optimistic Updating	24
12.1	What is Optimistic Updating?	24
12.2	Why Use It?	24
12.3	How It Works	24
12.4	Using useOptimistic Hook	24
12.5	Showing Loading State	25

12.6 Error Handling	26
12.7 Real-World Example	26
12.8 When to Use	26

Chapter 1

What are Form Actions?

1.1 Understanding Form Actions

Form Actions are a modern way to handle form submissions in web applications. Instead of managing form state and submission manually, Form Actions automate much of the process.

1.1.1 The Old Way vs New Way

Old Way (Using State):

- Create state for each field
- Create a handler for each field
- Track loading state manually
- Handle errors manually
- Send data to server manually

New Way (Using Form Actions):

- Write one function that does everything
- Framework handles the rest automatically
- Less code to write
- Less bugs
- Better performance

1.2 What is a Form Action Function?

A Form Action is a special function that:

1. Receives form data automatically
2. Validates the input
3. Processes the data

4. Updates the database or server
5. Returns a result
6. Updates the user interface

1.2.1 Key Benefit

Key Advantage: Form Actions handle server communication without page reload!

1.3 Where Form Actions Work

Form Actions are commonly used in:

- **Next.js** - Server Actions
- **Remix** - Actions
- **SvelteKit** - Form Actions
- Other modern frameworks

Remember: Form Actions simplify form handling significantly!

Chapter 2

Adding Validation Checks

2.1 Validation in Form Actions

Validation means checking if the user's input is correct before processing it.

2.2 Where to Validate

1. **Client-side** - Quick feedback to user
2. **Server-side** - Secure and reliable (always do this!)

2.3 Common Validations

- Is the field empty?
- Is email in correct format?
- Is password long enough?
- Does password match confirmation?
- Does the username already exist?
- Is the number within range?

2.4 Steps for Validation

1. Get the form data
2. Check each field
3. If invalid, return error message
4. If valid, continue processing

2.4.1 Example Validation Check

```
if (!email.includes('@')) {  
  return { error: 'Invalid email format' };  
}  
  
if (password.length < 8) {  
  return { error: 'Password too short' };  
}
```

Critical: Always validate on the server! Client validation can be disabled.

2.5 Returning Validation Errors

Form Actions return error objects that the UI can display:

```
return {  
  error: 'Email already exists',  
  field: 'email'  
};
```

Best Practice: Return clear, user-friendly error messages!

Chapter 3

Managing Form-dependent State with `useActionState()`

3.1 What is `useActionState()`?

`useActionState()` is a special hook that connects your form to a Form Action. It manages:

- Form data
- Loading state
- Errors
- Results

3.2 What It Returns

`useActionState()` returns four things:

Item	What It Does
State	Current state (errors or results)
FormAction	Function to pass to form
IsPending	Is the form being submitted?
StartTransition	Control the submission

3.3 Basic Usage

```
const [state, formAction, isPending] =  
  useActionState(submitForm, null);
```

3.4 How It Works

1. User submits form
2. `useActionState()` calls your action function
3. While waiting, `isPending` is true

4. When done, `state` has the result
5. UI updates automatically

Advantage: No manual state management needed!

3.5 Displaying Errors

```
{state?.error && (  
  <p className="error">{state.error}</p>  
)}
```

Tip: Show errors in a user-friendly way!

Chapter 4

Using User Input

4.1 Accessing Form Data

Form Actions automatically receive form data. You access it easily:

4.2 FormData Object

When a user submits a form, the browser creates a `FormData` object containing all the form fields.

4.2.1 Extracting Data

```
async function submitForm(formData) {  
  const email = formData.get('email');  
  const password = formData.get('password');  
  const name = formData.get('name');  
  
  // Now process the data  
}
```

4.3 Field Names Matter

Each form input must have a `name` attribute:

```
<input name="email" type="email" />  
<input name="password" type="password" />  
<textarea name="message"></textarea>
```

4.4 Getting Multiple Values

For checkboxes with the same name:

```
const interests = formData.getAll('interests');  
// Returns array of checked values
```

4.5 File Uploads

```
const file = formData.get('photo');  
const fileSize = file.size;  
const fileName = file.name;
```

Remember: Always give inputs meaningful names!

Chapter 5

Moving the Action Function out of the Component

5.1 Why Move the Action?

Keeping action functions in separate files helps because:

- Cleaner component code
- Easy to reuse the action
- Easy to test
- Separates concerns
- Better organization

5.2 How to Organize

File Structure:

```
/app
  /actions
    formActions.js    (Action functions)
    userActions.js
  /components
    LoginForm.jsx      (Component)
    SignupForm.jsx
```

5.3 Creating Action File

File: actions/formActions.js

```
'use server'

export async function submitLoginForm(formData) {
  const email = formData.get('email');
```

```
const password = formData.get('password');

// Validation
if (!email || !password) {
  return { error: 'All fields required' };
}

// Process
// ...

return { success: true };
}
```

5.4 Using in Component

```
import { submitLoginForm } from '@actions/formActions';

export function LoginForm() {
  const [state, formAction, isPending] =
    useActionState(submitLoginForm, null);

  return (
    <form action={formAction}>
      {/* form fields */}
    </form>
  );
}
```

Best Practice: Always keep actions in separate files!

5.5 The 'use server' Directive

'use server'

This tells the framework:

- This function runs on the server
- It's safe (hidden from users)
- It can access databases

Chapter 6

Handling Form Submission

6.1 Form Submission Flow

When user clicks submit button:

1. Browser collects form data
2. Creates FormData object
3. Calls the form action
4. Action processes data
5. Result returned to component
6. UI updates with result

6.2 Preventing Default Behavior

Form Actions automatically prevent page reload. No need to write:

```
event.preventDefault()
```

This happens automatically!

6.3 After Submission

Common things to do after submission:

- Show success message
- Clear the form
- Redirect to another page
- Update data on page
- Show error if failed

6.4 Example: Complete Flow

```
async function handleSubmit(formData) {  
  // 1. Validate  
  const email = formData.get('email');  
  if (!email) return { error: 'Email required' };  
  
  // 2. Process  
  const result = await saveToDatabase(email);  
  
  // 3. Return result  
  if (result.ok) {  
    return { success: 'Saved successfully!' };  
  } else {  
    return { error: 'Failed to save' };  
  }  
}
```

Tip: Always return something from your action!

Chapter 7

Working with Asynchronous Form Actions

7.1 Why Async?

Most form actions need to:

- Talk to a database
- Make API calls
- Process files
- Fetch from external services

All these take time!

7.2 Using Async Syntax

```
async function submitForm(formData) {  
  // Can use await here  
  const result = await saveData(formData);  
  return result;  
}
```

7.3 Waiting for Results

```
async function submitForm(formData) {  
  // Wait for database  
  const user = await db.user.create({  
    email: formData.get('email')  
  });  
  
  // Wait for email send  
  await sendEmail(user.email);  
}
```

```
    return { success: true, userId: user.id };  
  }  
}
```

7.4 Handling Errors

```
async function submitForm(formData) {  
  try {  
    const result = await saveData(formData);  
    return { success: true };  
  } catch (error) {  
    return { error: 'Something went wrong' };  
  }  
}
```

7.5 Multiple Operations

Sometimes you need to do several things:

```
async function submitForm(formData) {  
  // Do these in order  
  const user = await createUser(formData);  
  const profile = await createProfile(user.id);  
  const email = await sendWelcomeEmail(user.email);  
  
  return { success: true };  
}
```

Or do them in parallel:

```
async function submitForm(formData) {  
  // Do these at the same time  
  const [user, notification] = await Promise.all([  
    createUser(formData),  
    sendNotification(formData.get('email'))  
  ]);  
  
  return { success: true };  
}
```

Performance Tip: Use `Promise.all()` for parallel operations!

Chapter 8

Updating the UI with `useFormStatus()`

8.1 What is `useFormStatus()`?

`useFormStatus()` gives you information about the form submission:

- Is it pending (being submitted)?
- What data is being submitted?
- What action is running?

8.2 What It Returns

Property	What It Is
<code>pending</code>	true while form is submitting
<code>data</code>	<code>FormData</code> being submitted
<code>method</code>	HTTP method (POST, PUT, etc.)
<code>action</code>	Which action function

8.3 Common Use: Disable Submit Button

Show a loading state while submitting:

```
import { useFormStatus } from 'react-dom';

function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button disabled={pending}>
      {pending ? 'Saving...' : 'Submit'}
    </button>
  );
}
```

8.4 Show Loading Spinner

```
function FormContent() {
  const { pending } = useFormStatus();

  return (
    <>
      {pending && <Spinner />}
      <input name="email" />
    </>
  );
}
```

8.5 Disable All Form Fields

```
function MyForm() {
  const { pending } = useFormStatus();

  return (
    <fieldset disabled={pending}>
      <input name="email" />
      <input name="password" />
      <button>Submit</button>
    </fieldset>
  );
}
```

Best Practice: Always show loading state to users!

8.6 Important Note

`useFormStatus()` must be used in a child component of the form:

```
// This WORKS:
<form action={handleSubmit}>
  <SubmitButton />  {/* Can use useFormStatus */}
</form>
```

```
// This DOESN'T work:
<SubmitButton />    {/* Not in form */}
<form action={handleSubmit}></form>
```

Chapter 9

Registering Multiple Form Actions

9.1 One Form, Multiple Actions

Sometimes one form has multiple buttons that do different things:

- Save Draft
- Publish
- Delete

9.2 Using formAction Attribute

Each button can have its own action:

```
<form>
  <input name="title" />
  <textarea name="content"></textarea>

  <button formAction={saveDraft}>
    Save Draft
  </button>

  <button formAction={publish}>
    Publish
  </button>

  <button formAction={delete}>
    Delete
  </button>
</form>
```

9.3 Different Actions

```
async function saveDraft(formData) {
  // Save but don't publish
```

```
    return { success: 'Draft saved' };
  }

  async function publish(formData) {
    // Save and publish
    return { success: 'Published!' };
  }

  async function deletePost(formData) {
    // Delete everything
    return { success: 'Deleted' };
  }
```

9.4 Detecting Which Button

You can also detect which button was clicked:

```
const submitButton = formData.get('submitType');

if (submitButton === 'draft') {
  // Save draft
} else if (submitButton === 'publish') {
  // Publish
}
```

HTML:

```
<button name="submitType" value="draft">
  Save Draft
</button>

<button name="submitType" value="publish">
  Publish
</button>
```

Tip: Use `formAction` when buttons need different logic!

Chapter 10

Sending an HTTP Request via a Form Action

10.1 Making API Calls

Form Actions can communicate with backend APIs:

1. Process form data
2. Send to API endpoint
3. Get response
4. Return result to user

10.2 Using Fetch

```
async function submitForm(formData) {
  const email = formData.get('email');
  const password = formData.get('password');

  const response = await fetch('/api/login', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      email,
      password
    })
  });

  const data = await response.json();
  return data;
}
```

10.3 Error Handling

```
async function submitForm(formData) {
  try {
    const response = await fetch('/api/users', {
      method: 'POST',
      body: formData
    });

    if (!response.ok) {
      return { error: 'Server error' };
    }

    const data = await response.json();
    return { success: true, data };
  } catch (error) {
    return { error: 'Network error' };
  }
}
```

10.4 Different HTTP Methods

POST - Create new data:

```
const response = await fetch('/api/users', {
  method: 'POST',
  body: formData
});
```

PUT - Update data:

```
const response = await fetch('/api/users/123', {
  method: 'PUT',
  body: formData
});
```

DELETE - Remove data:

```
const response = await fetch('/api/users/123', {
  method: 'DELETE'
});
```

Important: Always validate API responses!

Chapter 11

Using the "pending" State from `useActionState()`

11.1 Accessing Pending State

`useActionState()` gives you a `isPending` boolean:

```
const [state, formAction, isPending] =  
  useActionState(submitForm, null);
```

11.2 What `isPending` Tells You

- `true` = Form is being submitted
- `false` = Form submission complete

11.3 Common Uses

11.3.1 Disable Submit Button

```
<button disabled={isPending}>  
  {isPending ? 'Submitting...' : 'Submit'}  
</button>
```

11.3.2 Show/Hide Spinner

```
{isPending && <Spinner />}
```

11.3.3 Disable Entire Form

```
<fieldset disabled={isPending}>  
  {/* form content */}  
</fieldset>
```

11.3.4 Show Different Message

```
<p>
  {isPending
    ? 'Saving your changes...'
    : 'Changes saved!'
  }
</p>
```

11.4 Pending vs useFormStatus

Both show pending state, but use them differently:

When to Use	Tool
In form component	useActionState
In child component	useFormStatus

Best Practice: Use `isPending` to provide user feedback!

Chapter 12

Adding Optimistic Updating

12.1 What is Optimistic Updating?

Optimistic updating means showing the result immediately, before the server confirms it.

12.2 Why Use It?

- Feels faster to users
- Better user experience
- Less waiting
- More responsive app

12.3 How It Works

1. User clicks submit
2. Show result immediately (optimistically)
3. Send request to server in background
4. If server confirms, keep showing result
5. If server rejects, show error and revert

12.4 Using useOptimistic Hook

```
import { useOptimistic } from 'react';

function MyForm() {
  const [list, setList] = useState([]);
  const [optimisticList, addOptimistic] =
    useOptimistic(list);

  async function handleSubmit(formData) {
```

```
const newItem = {
  id: Date.now(),
  text: formData.get('text')
};

// Show immediately
addOptimistic(newItem);

// Send to server
const result = await saveItem(newItem);

// Update with real data
setList([...list, result.item]);
}

return (
  <>
    <ul>
      {optimisticList.map(item => (
        <li key={item.id}>{item.text}</li>
      ))}
    </ul>
    <form action={handleSubmit}>
      <input name="text" />
      <button>Add</button>
    </form>
  </>
);
}
```

12.5 Showing Loading State

Mark optimistic items so users know they're pending:

```
{optimisticList.map(item => (
  <li key={item.id}
    className={item.pending ? 'pending' : ''}>
    {item.text}
  </li>
))}
```

CSS:

```
.pending {
  opacity: 0.6;
  font-style: italic;
}
```

12.6 Error Handling

If server rejects, the optimistic update reverts:

```
async function handleSubmit(formData) {
  const newItem = { text: formData.get('text') };

  addOptimistic(newItem);

  try {
    const result = await saveItem(newItem);
    setList([...list, result.item]);
  } catch (error) {
    // Optimistic update automatically reverts
    // Show error message
    alert('Failed to save');
  }
}
```

12.7 Real-World Example

Like-button on social media:

```
const [liked, setLiked] = useState(false);
const [optimisticLiked, addOptimistic] =
  useOptimistic(liked);

async function toggleLike() {
  addOptimistic(!liked);

  try {
    await fetch('/api/like', {
      method: 'POST'
    });
    setLiked(!liked);
  } catch (error) {
    // Reverts if error
  }
}
```

Advantage: Makes app feel super fast!

12.8 When to Use

Use optimistic updating for:

- Like/unlike buttons
- Adding items to lists

- Toggling switches
- Simple state changes

Don't use for:

- Complex operations
- Risky actions (delete)
- Financial transactions

Quick Summary

Form Actions Overview

Form Actions simplify form handling by:

1. Receiving form data automatically
2. Handling validation
3. Managing async operations
4. Returning results to UI
5. Reducing manual state management

Key Tools

Tool	Purpose
<code>useActionState()</code>	Connect form to action
<code>useFormStatus()</code>	Track submission state
<code>useOptimistic()</code>	Show result before confirmed
<code>FormData</code>	Access form fields

Best Practices

1. Keep actions in separate files
2. Always validate on the server
3. Show loading states
4. Handle errors gracefully
5. Use optimistic updating for simple changes
6. Return clear feedback to users
7. Use `async/await` for operations
8. Show user-friendly messages

Golden Rules

1. **Never trust client validation alone** - Always validate server-side
2. **Always provide feedback** - Tell users what's happening
3. **Use optimistic updates wisely** - Not for risky operations
4. **Keep forms simple** - Fewer fields = better UX
5. **Clear error messages** - Help users understand what went wrong

Quick Flow

```
User submits form
↓
Form Action called automatically
↓
Validate input
↓
Make async requests (DB, API)
↓
Return result
↓
useActionState updates UI
↓
User sees success or error
```

Form Actions make building modern web forms easier and faster!